

Демонстрация дискретной сети Хопфилда для восстановления изображений 🔗

 Open in Colab  Open in Kaggle

```
[15]: import numpy as np
import matplotlib.pyplot as plt

import brainpy as bp
import brainpy.math as bm
```

```
[16]: версия bp.____
```

```
[16]: '2.4.5.post4'
```

Математические основы дискретной сети Хопфилда описаны в [нашем предыдущем руководстве](#).

Данные для тестирования

Теперь мы загрузим данные. Я уже сохранил два изображения, с цифрами «9» и «4», чтобы упростить задачу. Мы также изменим их размер на исходный [28 x 28], чтобы потом можно было исказить их в этой области, а затем восстановить.

Данные можно скачать по этой ссылке:

https://github.com/brainpy/examples/blob/main/attractors/data/data_to_train_on.npy

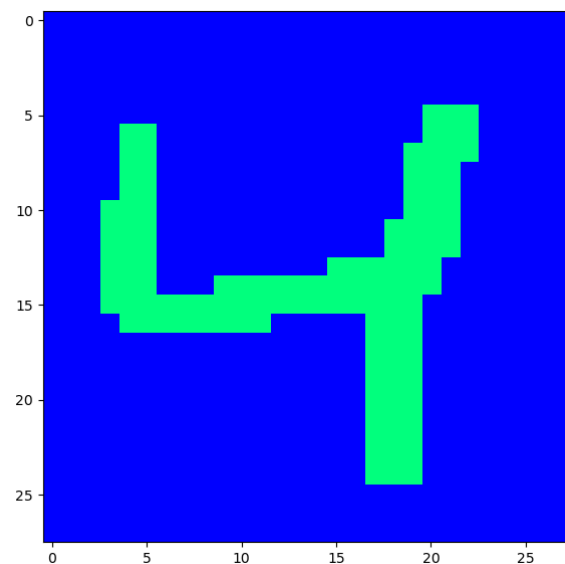
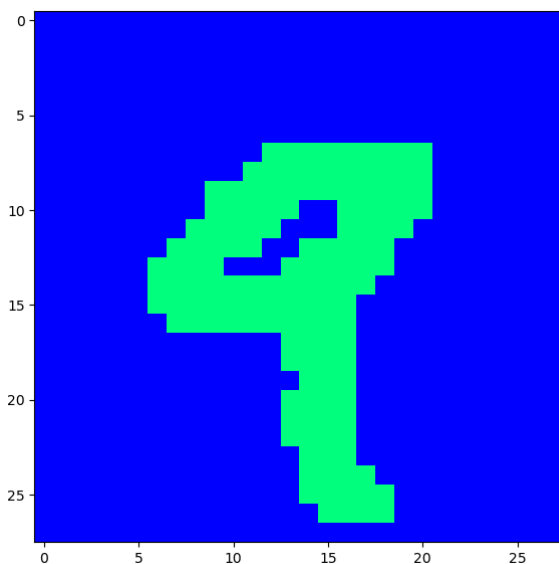
```
[17]: data = (np.load('data/data_to_train_on.npy')).astype(np.float32)

data.shape
```

```
[17]: (2, 784)
```

Как выглядят наши данные? Это всего лишь два бинарных изображения, на которых изображены цифры «9» и «4». Давайте посмотрим на них.

```
[18]: # Показываем обучающие данные – это всего лишь два разных образца, которые мы хотим
      # сохранить в сети Хопфилда.
fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(15,7.5))
ax[0].imshow(data[0].reshape(28, 28), interpolation='None', cmap='winter')
ax[1].imshow(data[1].reshape(28, 28), interpolation='None', cmap='winter')
plt.show()
```



С помощью этой функции мы возьмем хорошее изображение и искадим его, добавив два типа шума: шум Бернулли (когда каждый пиксель случайным образом меняет знак) или маскирующий шум, при котором мы закрашиваем целые участки изображения. Как мы увидим, наша сеть Хопфилда сможет восстановить исходное изображение путем минимизации энергии.

```
[19]: def add_bernouli_noise(img, prob=0.1, mask_value=1.):
      rand = np.random.rand(*img.shape) < prob
      return np.where(rand, mask_value, img)
```

```
[20]: def add_mask_noise(img, mask_value=-1, mask_matrix=np.zeros((2,2))):
      img = np.copy(img)
      img[mask_matrix[0,0]: mask_matrix[0,1], mask_matrix[1,0]: mask_matrix[1,1]] =
      mask_value
      return img
```

Модели

Мы используем нашу реализацию в [дискретной сети Хопфилда](#).

```

[21]: class HopfieldNet(bp.DynamicalSystem):
    def __init__(self, num):
        super().__init__()
        self.num = num
        self.weight = bm.Variable(bm.zeros([num, num]))

    # Функция обучения сети Хопфилда. (путем обновления весов)
    @bm.cls_jit
    def store_patterns(self, samples):
        # data: массив с [d, N]. 'd' – это количество выборок данных, используемых для
        обучения. (В данном случае 2)
        assert samples.ndim == 2
        assert samples.shape[1] == self.num
        # Перебираем все выборки данных.
        bm.for_loop(self.store, samples)

    # Сети Хопфилда – это разновидность рекуррентных нейронных сетей, хотя и без
    внутренних связей. Поэтому нам нужно убедиться, что диагональные элементы итоговой
    матрицы весов равны нулю.
    bm.fill_diagonal(self.weight, 0)

    # Сохранение одного образца
    @bm.cls_jit
    def store(self, sample):
        # sample – это массив размером (N,)
        assert self.num == sample.shape[0]

    # Перекрестное произведение данных дает правило обновления нейронного Хопфилда.
    w_update = bm.outer(выборка, sample)

    # Суммируем все перекрестные произведения шаблона.
    self.weight += w_update

    определение async_recover(self, sample, n, energy=False):
    # n: количество итераций для восстановления.
    # энергия: вычисление энергетической функции
    idxs = bm.random.randint(0, self.num, n) # выбранные позиции
    # для JIT-компиляции необходимо пометить изменяемое значение как Variable
    sample = bm.Variable(sample)

    def recover(i):
        # i: позиция для обновления
        # обновление
        sample[i] = bm.sign(bm.inner(self.weight[i], sample))
        # вернуть энергию
        если энергия:
            вернуть self.energy (образец)

    r = bm.for_loop (восстановить, idxs) # для цикла JIT
    вернуть (образец, r), если энергия еще образец

    определение sync_recover(self, sample, n, energy=False):
    # n: количество итераций для восстановления.
    # энергия: вычислите функцию энергии
    idxs = bm.arange(n) # время обновления
    # Для JIT-компиляции необходимо пометить изменяемое значение как Variable
    sample = bm.Variable(sample)

    def recover(i):
        # i: позиция для обновления
        # обновление
        sample.value = bm.sign(self.weight @ sample)
        # возврат энергии

```

```

if energy:
    return self.energy(sample)

r = bm.for_loop (восстановление, idxs) # для цикла JIT
возврат (sample, r), если энергия еще sample

# Эта функция вычисляет энергию сетей Хопфилда.
энергия def (собственная величина, x):
# x: [N] вектор данных
return bm.inner(- x @ self.weight, x)

```

Training

OK this is where the fun begins! We are now going to train our Hopfield net. The objective is simple: We want it to store those two patterns we are feeding it, such that a faithful reconstruction can be performed, when corrupted versions of either of those inputs is given later in the future.

```

[22]: # Train the Hopfield net!

# So there are as many neurons as pixels per pattern, and this is what this
computes.
net = HopfieldNet(num=data.shape[1])

# Great - we are all set - now we train!
net.store_patterns(data)

```

Now at this point, we have a trained weight matrix, encoded with the pattern '4' and pattern '9'. (We will see later on that Hopfield nets aren't that memory efficient, but it's still amazing what they are capable of). Now remember - the objective here is simple: We want it to store those two patterns we are feeding it, such that faithful reconstruction can later be performed, when corrupted versions of either of those inputs are presented. But does it work? Let's find out!).

Testing

We can corrupt our data in many ways - and here, we can add two types of noise: Bernouli noise, (where every pixel is flipped in sign with a certain probability) and/or masking noise, (where we simply blot out certain regions with one particular sign).

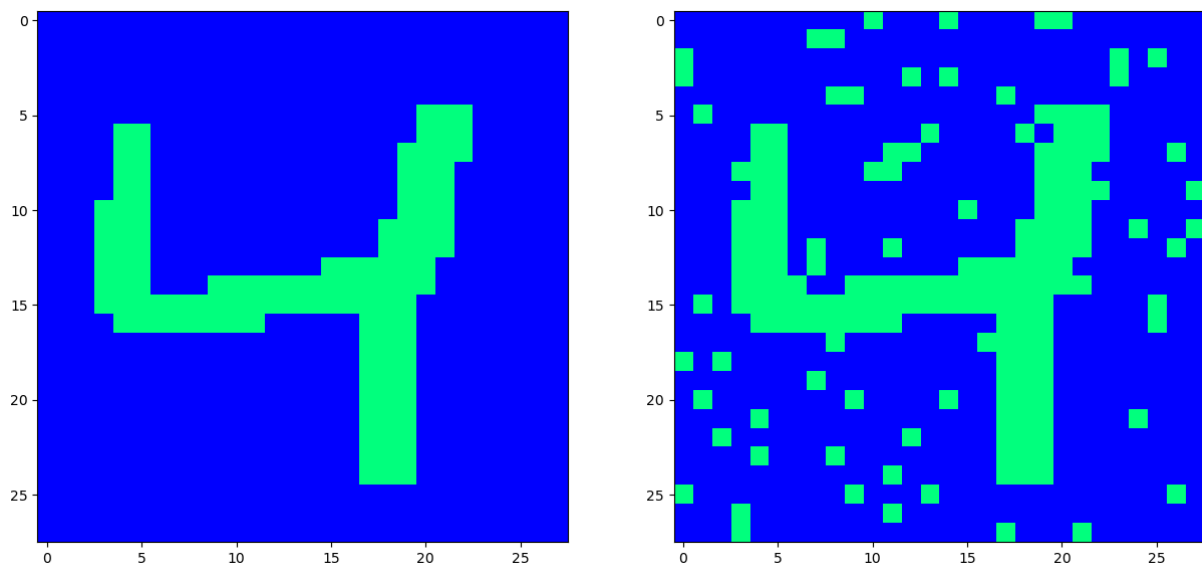
```

[23]: # Let's pick one the image '4', and corrupt it.
image = data[1]

# Corrupt the image with bernouli noise.
corrupted_image = add_bernouli_noise(image, prob=0.1)

# Let's see the original image, as well as the corrupted version.
fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(15,7.5))
ax[0].imshow(image.reshape(28, 28), interpolation='None', cmap='winter')
ax[1].imshow(corrupted_image.reshape(28, 28), interpolation='None', cmap='winte
plt.show()

```



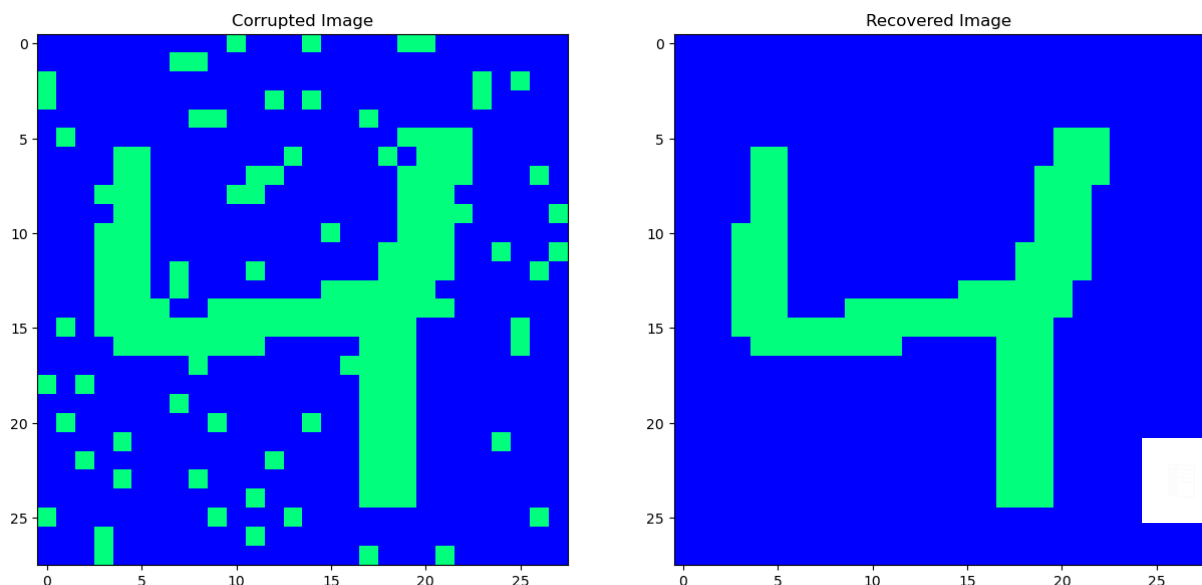
This is the moment of truth: With a corrupted input now at hand, we then present this to our trained net, and hope that it denoises it. We do so by picking pixels at random, and then evaluating their activation functions, based on the current input configuration, (corrupted image), and the set weights, (that do not change here, as the net has already been trained).

Since the Hopfield net stores patterns as minimal energy states, every iteration where we update a neuron/pixel should place the net into a lower energy configuration - that is - closer to the original pattern we stored - the weights are designed to do so after all.

```
[24]: # The number of iterations where we randomly update neurons/pixels
n_iterations = 5000

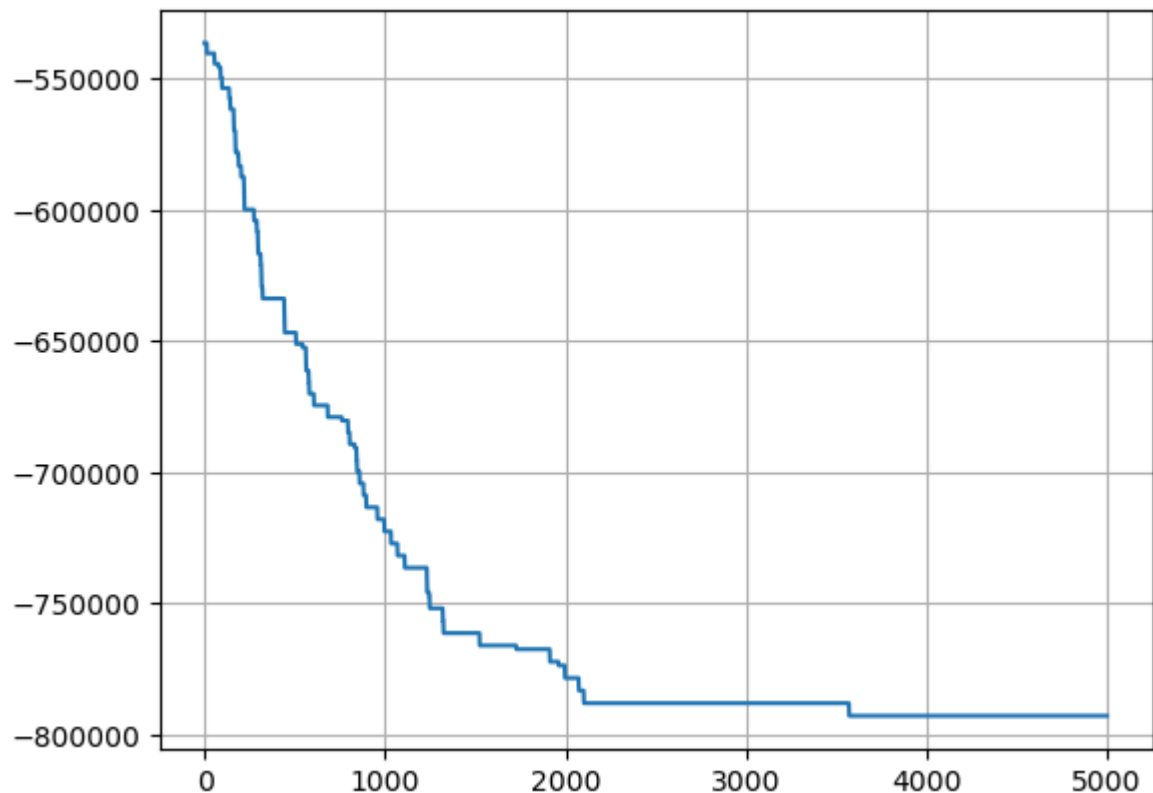
# Loop through, and recover the image from it's corrupted self.
# energy_vec: This will store the energy of the Hopfield net.
cleaned_image, energy_vec = net.async_recover(corrupted_image, n_iterations, True)

# (For imaging purposes)
fig, ax = plt.subplots(nrows = 1, ncols = 2, figsize=(15, 7.5))
ax[0].imshow(corrupted_image.reshape(28, 28), interpolation='None', cmap='winter')
ax[0].set_title('Corrupted Image')
ax[1].imshow(cleaned_image.reshape(28, 28), interpolation='None', cmap='winter')
ax[1].set_title('Recovered Image')
plt.show()
```



The Hopfield energy plotted as the recovery iteration index is shown below. We can see that as the recovery process was being run, the total net energy was being decreased.

```
[25]: # Plot the Hopfield energy during recovery
plt.plot(bm.as_numpy(energy_vec))
plt.grid('on')
plt.show()
```

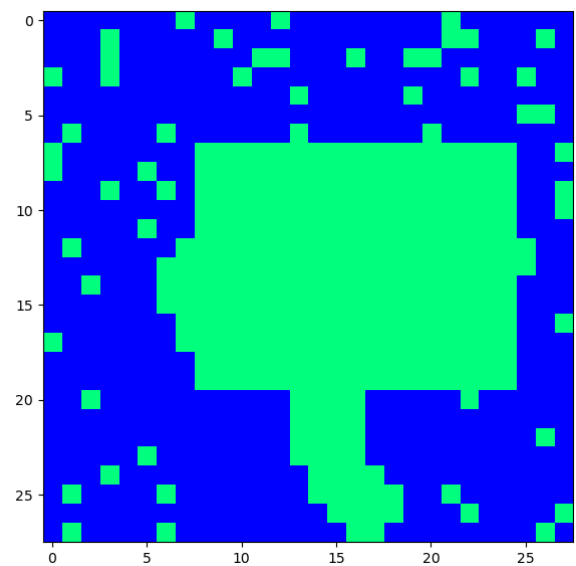
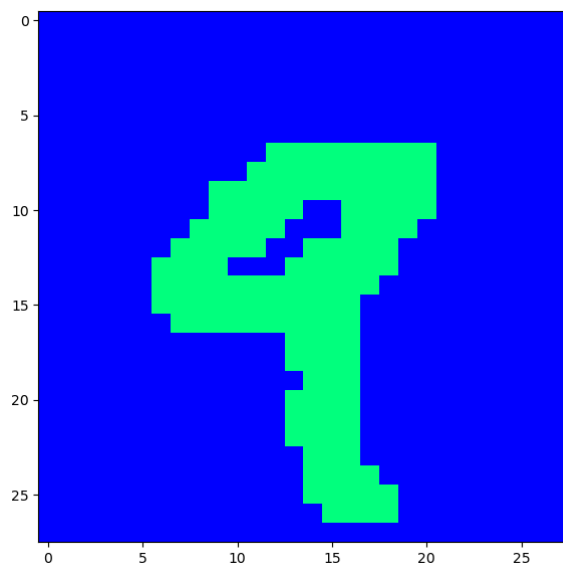


Our original pattern has been very nicely recovered. What about masking noise and bernouli noise? Let's try that.

```
[26]: # Masking and Bernouli noise
mask_matrix = np.asarray([[7, 20], [8, 25]], dtype=np.int32)

image = data[0].reshape(28, 28)
corrupted_image = add_mask_noise(image, mask_value=1, mask_matrix=mask_matrix)
corrupted_image = add_bernouli_noise(corrupted_image, prob=0.1)

fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(15,7.5))
ax[0].imshow(image, interpolation='None', cmap='winter')
ax[1].imshow(corrupted_image, interpolation='None', cmap='winter')
plt.show()
```

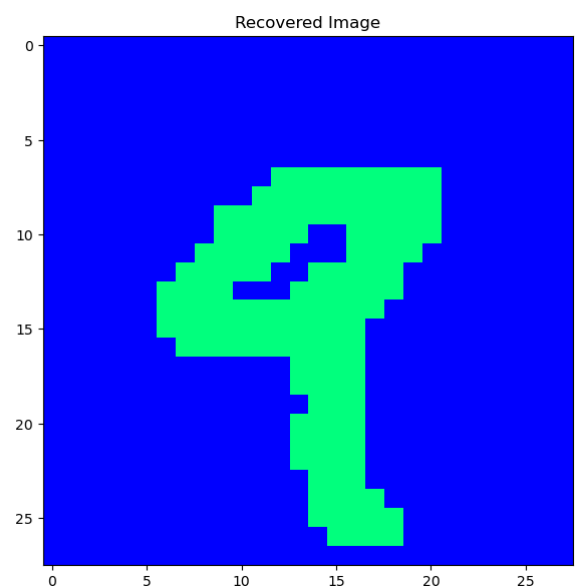
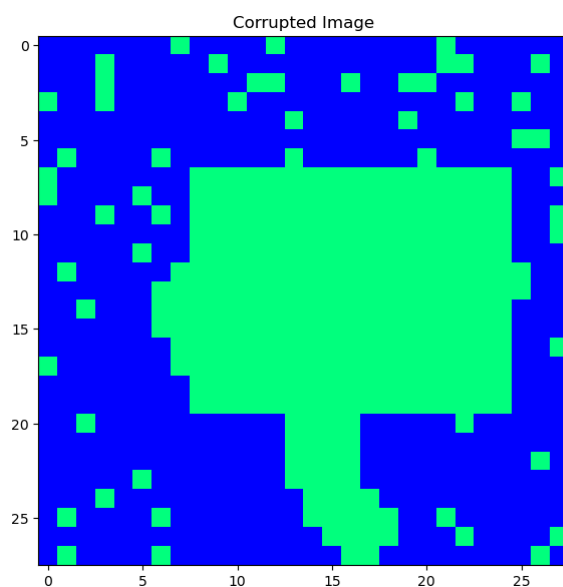


Now that's one ugly looking '9'. Can we recover the hidden pattern?

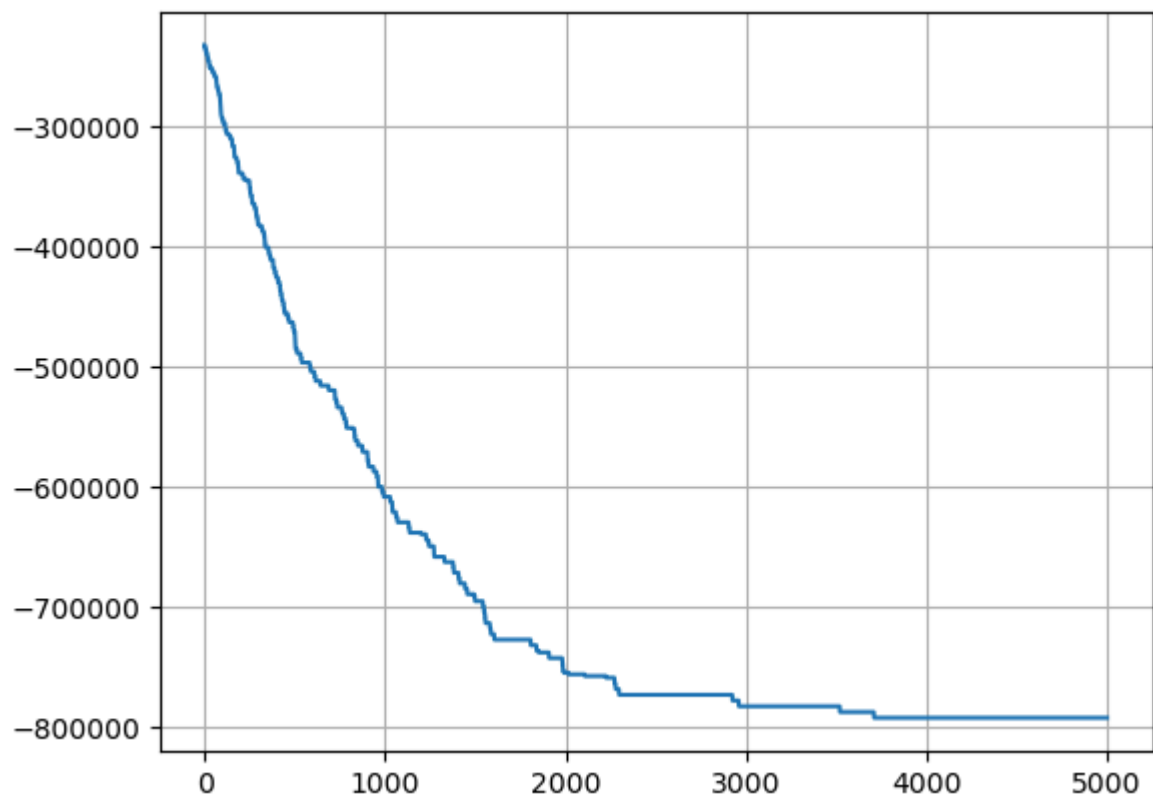
```
[27]: # The number of iterations where we randomly update neurons/pixels
n_iterations = 5000

# Loop through, and recover the image from it's corrupted self.
# energy_vec: This will store the energy of the Hopfield net.
cleaned_image, energy_vec = net.async_recover(corrupted_image.flatten(),
n_iterations, True)

# (For imaging purposes)
fig, ax = plt.subplots(nrows = 1, ncols = 2, figsize=(15, 7.5))
ax[0].imshow(corrupted_image.reshape(28, 28), interpolation='None', cmap='winter')
ax[0].set_title('Corrupted Image')
ax[1].imshow(cleaned_image.reshape(28, 28), interpolation='None', cmap='winter')
ax[1].set_title('Recovered Image')
plt.show()
```



```
[28]: # Plot the Hopfield energy during recovery
plt.plot(bm.as_numpy(energy_vec))
plt.grid('on')
plt.show()
```



Now this is exactly what it means to be 'content addressable' - the fact that we are able to 'snap back' to a previous pattern among many patterns, starting from a corrupted variant was perhaps one of the first success stories of the associationist movement - Hopfield nets - totally connected RNNs, where the relationships between neurons/pixels are impressed into the weights as minimum energy states. In doing so, the iterative changing of neural activations becomes exactly like finding a configuration of least energy - something that is found in nature all the time.